Introduction to Machine Learning
Unit 7 Problems: Gradient Calculations and Nonlinear Optimization

Prof. Sundeep Rangan

1. *Simple gradient calculation.* Consider a function,

$$J = z_1 e^{z_1 z_2}, \quad z_1 = a_1 w_1 w_2, \quad z_2 = a_2 w_1 + a_3 w_2^2,$$

   (a) Compute the partial derivatives, $\partial J/\partial w_j$ for $j = 1, 2$.

   (b) Write a python function that, given $\mathbf{w}$ and $\mathbf{a}$ computes $J(\mathbf{w})$ and $\nabla J(\mathbf{w})$.

   Solution:

   (a) Using the chain rule:

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial z_1}\frac{\partial z_1}{\partial w_1} + \frac{\partial J}{\partial z_2}\frac{\partial z_2}{\partial w_1}$$

   First, compute the partial derivatives:

$$\frac{\partial J}{\partial z_1} = e^{z_1 z_2} + z_1 e^{z_1 z_2} z_2 = e^{z_1 z_2}(1 + z_1 z_2) \tag{1}$$

$$\frac{\partial J}{\partial z_2} = z_1 e^{z_1 z_2} z_1 = z_1^2 e^{z_1 z_2} \tag{2}$$

$$\frac{\partial z_1}{\partial w_1} = a_1 w_2 \tag{3}$$

$$\frac{\partial z_1}{\partial w_2} = a_1 w_1 \tag{4}$$

$$\frac{\partial z_2}{\partial w_1} = a_2 \tag{5}$$

$$\frac{\partial z_2}{\partial w_2} = 2 a_3 w_2 \tag{6}$$

   Therefore:

$$\frac{\partial J}{\partial w_1} = e^{z_1 z_2}(1 + z_1 z_2) \cdot a_1 w_2 + z_1^2 e^{z_1 z_2} \cdot a_2 = e^{z_1 z_2}[a_1 w_2 (1 + z_1 z_2) + a_2 z_1^2]$$

$$\frac{\partial J}{\partial w_2} = e^{z_1 z_2}(1 + z_1 z_2) \cdot a_1 w_1 + z_1^2 e^{z_1 z_2} \cdot 2 a_3 w_2 = e^{z_1 z_2}[a_1 w_1 (1 + z_1 z_2) + 2 a_3 w_2 z_1^2]$$

(b)
```python
def Jeval(w, a):
    w1, w2 = w[0], w[1]
    a1, a2, a3 = a[0], a[1], a[2]

    # Compute intermediate variables
    z1 = a1 * w1 * w2
    z2 = a2 * w1 + a3 * w2**2

    # Compute J
    J = z1 * np.exp(z1 * z2)

    # Compute gradients
    exp_term = np.exp(z1 * z2)
    grad_w1 = exp_term * (a1 * w2 * (1 + z1 * z2) + a2 * z1**2)
    grad_w2 = exp_term * (a1 * w1 * (1 + z1 * z2) + 2 * a3 * w2 * z1**2)

    return J, np.array([grad_w1, grad_w2])
```

2. *Gradient with a logarithmic loss.* Consider the loss function,

$$J(\mathbf{w}, b) := \sum_{i=1}^{N} (\log(y_i) - \log(\hat{y}_i))^2, \quad \hat{y}_i = \sum_{j=1}^{p} x_{ij} w_j + b,$$

This is an MSE loss function, but in log domain.

(a) Find the gradient components, $\partial J/\partial w_j$ and $\partial J/\partial b$.

(b) Complete the following python function

```python
def Jeval(w,b,...):
    ...
    return J, Jgradw, Jgradb
```

that computes J and $\nabla_w J$ and $\nabla_b J$. You need to complete the arguments of the function. To receive full credit, avoid using for loops.

Solution:

(a) Let $r_i = \log(y_i) - \log(\hat{y}_i)$. Then $J = \sum_{i=1}^{N} r_i^2$.
For $\frac{\partial J}{\partial w_j}$:

$$\frac{\partial J}{\partial w_j} = \sum_{i=1}^{N} 2 r_i \frac{\partial r_i}{\partial w_j}$$

Since $r_i = \log(y_i) - \log(\hat{y}_i)$ and $\hat{y}_i = \sum_{k=1}^{p} x_{ik} w_k + b$:

$$\frac{\partial r_i}{\partial w_j} = -\frac{1}{\hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_j} = -\frac{x_{ij}}{\hat{y}_i}$$

Therefore:

$$\frac{\partial J}{\partial w_j} = \sum_{i=1}^{N} 2r_i \left(-\frac{x_{ij}}{\hat{y}_i}\right) = -2\sum_{i=1}^{N} \frac{r_i x_{ij}}{\hat{y}_i}$$

For $\frac{\partial J}{\partial b}$:

$$\frac{\partial J}{\partial b} = \sum_{i=1}^{N} 2r_i \frac{\partial r_i}{\partial b} = \sum_{i=1}^{N} 2r_i \left(-\frac{1}{\hat{y}_i}\right) = -2\sum_{i=1}^{N} \frac{r_i}{\hat{y}_i}$$

(b)
```python
def Jeval(w, b, X, y):
    # Compute predictions
    yhat = X @ w + b

    # Compute residuals in log domain
    r = np.log(y) - np.log(yhat)

    # Compute loss
    J = np.sum(r**2)

    # Compute gradients
    Jgradw = -2 * np.sum((r / yhat)[:, np.newaxis] * X, axis=0)
    Jgradb = -2 * np.sum(r / yhat)

    return J, Jgradw, Jgradb
```

3. *Gradient with an inverse function.* Consider the nonlinear least squares fit loss funciton

$$J(\mathbf{w}) = \sum_{i=1}^{n} \left[y_i - \frac{1}{w_0 + \sum_{j=1}^{d} w_j x_{ij}} \right]^2.$$

(a) Compute the gradient components, $\partial J/\partial w_j$. You may want to define the intermediate variable,

$$z_i = w_0 + \sum_{j=1}^{d} w_j x_{ij}.$$

Also, you can write separate answers for $\partial J/\partial w_0$ and $\partial J/w_j$ for $j = 1, \ldots, d$.

(b) Complete the following function to compute the loss and gradient,

```python
def Jeval(w,...):
    ...
    return J, Jgrad
```

For the gradient, you may wish to use the function,

```python
Jgrad = np.hstack((Jgrad0, Jgrad1))
```

to stack two vectors.

Solution:

(a) Let $z_i = w_0 + \sum_{j=1}^{d} w_j x_{ij}$ and $r_i = y_i - \frac{1}{z_i}$.
Then $J = \sum_{i=1}^{n} r_i^2$.
For $\frac{\partial J}{\partial w_0}$:

$$\frac{\partial J}{\partial w_0} = \sum_{i=1}^{n} 2 r_i \frac{\partial r_i}{\partial w_0}$$

Since $r_i = y_i - \frac{1}{z_i}$ and $\frac{\partial z_i}{\partial w_0} = 1$:

$$\frac{\partial r_i}{\partial w_0} = -\frac{\partial}{\partial w_0}\left(\frac{1}{z_i}\right) = \frac{1}{z_i^2}\frac{\partial z_i}{\partial w_0} = \frac{1}{z_i^2}$$

Therefore:

$$\frac{\partial J}{\partial w_0} = \sum_{i=1}^{n} 2 r_i \frac{1}{z_i^2} = 2\sum_{i=1}^{n} \frac{r_i}{z_i^2}$$

For $\frac{\partial J}{\partial w_j}$ where $j = 1, \ldots, d$:

$$\frac{\partial J}{\partial w_j} = \sum_{i=1}^{n} 2 r_i \frac{\partial r_i}{\partial w_j}$$

Since $\frac{\partial z_i}{\partial w_j} = x_{ij}$:

$$\frac{\partial r_i}{\partial w_j} = -\frac{\partial}{\partial w_j}\left(\frac{1}{z_i}\right) = \frac{1}{z_i^2}\frac{\partial z_i}{\partial w_j} = \frac{x_{ij}}{z_i^2}$$

Therefore:

$$\frac{\partial J}{\partial w_j} = \sum_{i=1}^{n} 2 r_i \frac{x_{ij}}{z_i^2} = 2\sum_{i=1}^{n} \frac{r_i x_{ij}}{z_i^2}$$

(b)
```
def Jeval(w, X, y):
    w0 = w[0]
    w1 = w[1:]

    # Compute z_i = w_0 + sum(w_j * x_{ij})
    z = w0 + X @ w1

    # Compute residuals
    r = y - 1/z

    # Compute loss
    J = np.sum(r**2)

    # Compute gradients
    Jgrad0 = 2 * np.sum(r / z**2)
    Jgrad1 = 2 * np.sum((r / z**2)[:, np.newaxis] * X, axis=0)
```

```
    # Stack gradients
    Jgrad = np.hstack((Jgrad0, Jgrad1))

    return J, Jgrad
```

4. *Gradient with nonlinear parametrization.* Given data (x_i, y_i) with binary class labels $y_i \in \{0, 1\}$, consider the binary cross-entropy loss function,

$$J(\mathbf{a}, \mathbf{b}) := \sum_{i=1}^{N} \log(1 + e^{z_i}) - y_i z_i, \quad z_i = \sum_{j=1}^{d} a_j e^{-(x_i - b_j)^2/2}.$$

   (a) Compute the gradient components, $\partial J/\partial a_j$ and $\partial J/\partial b_j$.

   (b) Complete the following function to compute the loss and gradient,

```
    def Jeval(a,b,...):
        ...
        return J, Jgrada, Jgradb
```

   Avoid for loops to receive full credit.

Solution:

(a) Let $J = \sum_{i=1}^{N} [\log(1 + e^{z_i}) - y_i z_i]$ where $z_i = \sum_{j=1}^{d} a_j e^{-(x_i - b_j)^2/2}$.

For $\frac{\partial J}{\partial a_j}$:

$$\frac{\partial J}{\partial a_j} = \sum_{i=1}^{N} \left[\frac{e^{z_i}}{1 + e^{z_i}} \frac{\partial z_i}{\partial a_j} - y_i \frac{\partial z_i}{\partial a_j} \right]$$

Since $\frac{\partial z_i}{\partial a_j} = e^{-(x_i - b_j)^2/2}$:

$$\frac{\partial J}{\partial a_j} = \sum_{i=1}^{N} \left[\frac{e^{z_i}}{1 + e^{z_i}} - y_i \right] e^{-(x_i - b_j)^2/2}$$

For $\frac{\partial J}{\partial b_j}$:

$$\frac{\partial J}{\partial b_j} = \sum_{i=1}^{N} \left[\frac{e^{z_i}}{1 + e^{z_i}} \frac{\partial z_i}{\partial b_j} - y_i \frac{\partial z_i}{\partial b_j} \right]$$

Since $\frac{\partial z_i}{\partial b_j} = a_j e^{-(x_i - b_j)^2/2} \cdot (x_i - b_j)$:

$$\frac{\partial J}{\partial b_j} = \sum_{i=1}^{N} \left[\frac{e^{z_i}}{1 + e^{z_i}} - y_i \right] a_j e^{-(x_i - b_j)^2/2} (x_i - b_j)$$

(b)
```
def Jeval(a, b, x, y):
    # Compute z_i for all i
    # x: (N,), a: (d,), b: (d,)
    # Broadcasting: (N,1) — (1,d) = (N,d)
    diff = x[:, np.newaxis] — b[np.newaxis, :]  # (N,d)
    exp_term = np.exp(—diff**2 / 2)  # (N,d)
    z = np.sum(a * exp_term, axis=1)  # (N,)

    # Compute sigmoid and loss
    sigmoid = 1 / (1 + np.exp(—z))  # (N,)
    J = np.sum(np.log(1 + np.exp(z)) — y * z)

    # Compute gradients
    residual = sigmoid — y  # (N,)

    # Gradient w.r.t. a
    Jgrada = np.sum(residual[:, np.newaxis] * exp_term, axis=0)  # (d,)

    # Gradient w.r.t. b
    Jgradb = np.sum(residual[:, np.newaxis] * a * exp_term * diff, axis=0)  # (d,)

    return J, Jgrada, Jgradb
```

5. *Finding local and global minima.* Consider the function

$$f(x) = \frac{1}{4}x^2 + 1 - \cos(2\pi x).$$

(a) Approximately draw $f(x)$.

(b) Write an equation for the gradient descent update to minimize $f(x)$.

(c) Using the graph in part (a), where is the global minima of $f(x)$?

(d) Using the graph in part (a), find one initial condition where gradient descent could end up converging to a local minima that is not the global minima. The local minima do not have a closed form expression, but you should be able to use the graph in part (a) to "eyeball" an initial condition close to a bad local minima.

Solution:

(a) The function $f(x) = \frac{1}{4}x^2 + 1 - \cos(2\pi x)$ has:

- A quadratic term $\frac{1}{4}x^2$ that creates a parabola centered at $x = 0$
- A periodic term $-\cos(2\pi x)$ with period 1 that oscillates between -1 and 1
- The function will have multiple local minima due to the cosine term

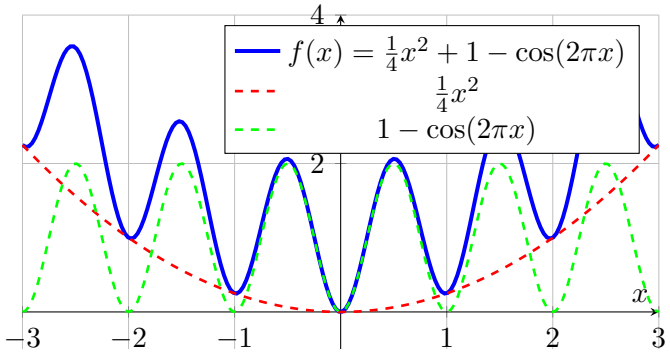

Figure 1: Plot of $f(x) = \frac{1}{4}x^2 + 1 - \cos(2\pi x)$ showing the global minimum at $x = 0$ and multiple local minima due to the periodic cosine term.

The graph shows a parabola with periodic oscillations. The global minimum is at $x = 0$ where $f(0) = 1 - \cos(0) = 0$.

(b) The gradient is:

$$f'(x) = \frac{1}{2}x + 2\pi \sin(2\pi x)$$

The gradient descent update is:

$$x^{k+1} = x^k - \alpha f'(x^k) = x^k - \alpha \left(\frac{1}{2}x^k + 2\pi \sin(2\pi x^k) \right)$$

(c) The global minimum is at $x = 0$, where $f(0) = 0$.

(d) A good initial condition that could lead to a local minimum (not global) would be around $x = \pm 1$ or $x = \pm 2$. For example, starting at $x = 1$:

- At $x = 1$: $f'(1) = \frac{1}{2} + 2\pi \sin(2\pi) = \frac{1}{2} > 0$
- The gradient is positive, so gradient descent will move left
- It could converge to a local minimum around $x \approx 0.5$ or $x \approx -0.5$

Another example: starting at $x = 2$:

- At $x = 2$: $f'(2) = 1 + 2\pi \sin(4\pi) = 1 > 0$
- The gradient is positive, so gradient descent will move left
- It could converge to a local minimum around $x \approx 1.5$ or $x \approx 0.5$

6. In this problem, we will see why gradient descent can often exhibit very slow convergence, even on apparently simple functions. Consider the objective function,

$$J(\mathbf{w}) = \frac{1}{2}b_1 w_1^2 + \frac{1}{2}b_2 w_2^2,$$

defined on a vector $\mathbf{w} = (w_1, w_2)$ with constants $b_2 > b_1 > 0$.

(a) What is the gradient $\nabla J(\mathbf{w})$?

(b) What is the minimum $\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w})$?

(c) Part (b) shows that we can minimize $J(\mathbf{w})$ easily by hand. But, suppose we tried to minimize it via gradient descent. Show that the gradient descent update of $\mathbf{w}$ with a step-size α has the form,

$$w_1^{k+1} = \rho_1 w_1^k, \quad w_2^{k+1} = \rho_2 w_2^k,$$

for some constants ρ_i, $i = 1, 2$. Write ρ_i in terms of b_i and the step-size α.

(d) For what values α will gradient descent converge to the minimum? That is, what step sizes guarantee that $\mathbf{w}^k \to \mathbf{w}^*$.

(e) Take $\alpha = 2/(b_1 + b_2)$. It can be shown that this choice of α results in the fastest convergence. You do not need to show this. But, show that with this selection of α,

$$\|\mathbf{w}^k\| = C^k \|\mathbf{w}^0\|, \quad C = \frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{b_2}{b_1}.$$

The term κ is called the *condition number*. The above calculation shows that when κ is very large, $C \approx 1$ and the convergence of gradient descent is slow. In general, gradient descent performs poorly when the problems are ill-conditioned like this.

Solution:

(a) The gradient is:

$$\nabla J(\mathbf{w}) = \begin{bmatrix} b_1 w_1 \\ b_2 w_2 \end{bmatrix}$$

(b) The minimum occurs where the gradient is zero:

$$\nabla J(\mathbf{w}^*) = \begin{bmatrix} b_1 w_1^* \\ b_2 w_2^* \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Therefore $\mathbf{w}^* = (0, 0)$.

(c) The gradient descent update is:

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \alpha \nabla J(\mathbf{w}^k) = \mathbf{w}^k - \alpha \begin{bmatrix} b_1 w_1^k \\ b_2 w_2^k \end{bmatrix}$$

This gives:

$$w_1^{k+1} = w_1^k - \alpha b_1 w_1^k = (1 - \alpha b_1) w_1^k = \rho_1 w_1^k$$
$$w_2^{k+1} = w_2^k - \alpha b_2 w_2^k = (1 - \alpha b_2) w_2^k = \rho_2 w_2^k$$

where $\rho_1 = 1 - \alpha b_1$ and $\rho_2 = 1 - \alpha b_2$.

(d) For convergence, we need $|\rho_i| < 1$ for $i = 1, 2$.
For $\rho_1 = 1 - \alpha b_1$: we need $-1 < 1 - \alpha b_1 < 1$, which gives $0 < \alpha < \frac{2}{b_1}$.
For $\rho_2 = 1 - \alpha b_2$: we need $-1 < 1 - \alpha b_2 < 1$, which gives $0 < \alpha < \frac{2}{b_2}$.
Since $b_2 > b_1$, the more restrictive condition is $\alpha < \frac{2}{b_2}$.
Therefore, gradient descent converges when $0 < \alpha < \frac{2}{b_2}$.

(e) With $\alpha = \frac{2}{b_1+b_2}$:

$$\rho_1 = 1 - \frac{2b_1}{b_1+b_2} = \frac{b_1+b_2-2b_1}{b_1+b_2} = \frac{b_2-b_1}{b_1+b_2}$$

$$\rho_2 = 1 - \frac{2b_2}{b_1+b_2} = \frac{b_1+b_2-2b_2}{b_1+b_2} = \frac{b_1-b_2}{b_1+b_2} = -\frac{b_2-b_1}{b_1+b_2}$$

Since $b_2 > b_1$, we have $\rho_1 = \frac{b_2-b_1}{b_1+b_2}$ and $\rho_2 = -\frac{b_2-b_1}{b_1+b_2}$.
The convergence rate is determined by the larger of $|\rho_1|$ and $|\rho_2|$, which is:

$$C = \frac{b_2-b_1}{b_1+b_2} = \frac{b_2/b_1-1}{b_2/b_1+1} = \frac{\kappa-1}{\kappa+1}$$

where $\kappa = \frac{b_2}{b_1}$ is the condition number.
The norm decreases as:

$$\|\mathbf{w}^k\| = C^k\|\mathbf{w}^0\|$$

When κ is very large (ill-conditioned), $C \approx 1$ and convergence is slow.

7. *Matrix minimization.* Consider the problem of finding a matrix $\mathbf{P} \in \mathbb{R}^{m\times m}$ to minimize the loss function,

$$J(\mathbf{P}) = \sum_{i=1}^{n} \left[\frac{z_i}{y_i} - \ln(z_i)\right], \quad z_i = \mathbf{x}_i^\mathsf{T}\mathbf{P}\mathbf{x}_i.$$

The problem arises in wireless communications where an m-antenna receiver wishes to estimate a spatial covariance matrix $\mathbf{P}$ from n power measurements. In this setting, $y_i > 0$ is the i-th receive power measurement and $\mathbf{x}_i$ is the beamforming direction for that measurement. In reality, the quantities would be complex, but for simplicity we will just look at the real-valued case. See the following article for more details:

> Eliasi, Parisa A., Sundeep Rangan, and Theodore S. Rappaport. "Low-rank spatial channel estimation for millimeter wave cellular systems," *IEEE Transactions on Wireless Communications* 16.5 (2017): 2748-2759.

(a) What is the gradient $\nabla_{\mathbf{P}} z_i$?

(b) What is the gradient $\nabla_{\mathbf{P}} J(\mathbf{P})$?

(c) Write a few lines of python code to evaluate $J(\mathbf{P})$ and $\nabla_{\mathbf{P}} J(\mathbf{P})$ given data $\mathbf{x}_i$ and y_i. You can use a for loop.

(d) See if you can rewrite (c) without a for loop. You will need Python broadcasting.

Solution:

(a) For $z_i = \mathbf{x}_i^T\mathbf{P}\mathbf{x}_i$, we have:

$$\nabla_{\mathbf{P}} z_i = \mathbf{x}_i\mathbf{x}_i^T$$

This can be derived using the matrix derivative formula $\frac{\partial}{\partial \mathbf{P}}(\mathbf{x}^T\mathbf{P}\mathbf{x}) = \mathbf{x}\mathbf{x}^T$.

(b) The gradient of $J(\mathbf{P})$ is:

$$\nabla_{\mathbf{P}} J(\mathbf{P}) = \sum_{i=1}^{n} \left[\frac{1}{y_i} - \frac{1}{z_i}\right] \nabla_{\mathbf{P}} z_i = \sum_{i=1}^{n} \left[\frac{1}{y_i} - \frac{1}{z_i}\right] \mathbf{x}_i \mathbf{x}_i^T$$

(c)
```
def Jeval(P, X, y):
    n, m = X.shape
    J = 0
    grad_P = np.zeros((m, m))

    for i in range(n):
        z_i = X[i] @ P @ X[i]
        J += z_i / y[i] - np.log(z_i)
        grad_P += (1/y[i] - 1/z_i) * np.outer(X[i], X[i])

    return J, grad_P
```

(d)
```
def Jeval_vectorized(P, X, y):
    # X: (n, m), P: (m, m), y: (n,)
    z = np.sum(X @ P * X, axis=1)  # (n,) - element-wise multiplication
    J = np.sum(z / y - np.log(z))

    # Compute gradients using broadcasting
    coeffs = (1/y - 1/z)[:, np.newaxis, np.newaxis]  # (n, 1, 1)
    X_outer = X[:, :, np.newaxis] * X[:, np.newaxis, :]  # (n, m, m)
    grad_P = np.sum(coeffs * X_outer, axis=0)  # (m, m)

    return J, grad_P
```

8. *Nested optimization.* Suppose we are given a loss function $J(\mathbf{w}_1, \mathbf{w}_2)$ with two parameter vectors $\mathbf{w}_1$ and $\mathbf{w}_2$. In some cases, it is easy to minimize over one of the sets of parameters, say $\mathbf{w}_2$, while holding the other parameter vector (say, $\mathbf{w}_1$) constant. In this case, one could perform the following *nested* minimization: Define

$$J_1(\mathbf{w}_1) := \min_{\mathbf{w}_2} J(\mathbf{w}_1, \mathbf{w}_2), \quad \widehat{\mathbf{w}}_2(\mathbf{w}_1) := \arg\min_{\mathbf{w}_2} J(\mathbf{w}_1, \mathbf{w}_2),$$

which represent the minimum and argument of the loss function over $\mathbf{w}_2$ holding $\mathbf{w}_1$ constant. Then,

$$\widehat{\mathbf{w}}_1 = \arg\min_{\mathbf{w}_1} J_1(\mathbf{w}_1) = \arg\min_{\mathbf{w}_1} \min_{\mathbf{w}_2} J(\mathbf{w}_1, \mathbf{w}_2).$$

Hence, we can find the optimal $\mathbf{w}_1$ by minimizing $J_1(\mathbf{w}_1)$ instead of minimizing $J(\mathbf{w}_1, \mathbf{w}_2)$ over $\mathbf{w}_1$ and $\mathbf{w}_2$.

(a) Show that the gradient of $J_1(\mathbf{w}_1)$ is given by

$$\nabla_{\mathbf{w}_1} J_1(\mathbf{w}_1) = \nabla_{\mathbf{w}_1} J(\mathbf{w}_1, \mathbf{w}_2)|_{\mathbf{w}_2 = \widehat{\mathbf{w}}_2}.$$

Thus, given $\mathbf{w}_1$, we can evaluate the gradient from (i) solve the minimization $\widehat{\mathbf{w}}_2 := \arg\min_{\mathbf{w}_2} J(\mathbf{w}_1, \mathbf{w}_2)$; and (ii) take the gradient $\nabla_{\mathbf{w}_1} J(\mathbf{w}_1, \mathbf{w}_2)$ and evaluate at $\mathbf{w}_2 = \widehat{\mathbf{w}}_2$.

(b) Suppose we want to minimize a nonlinear least squares,

$$J(\mathbf{a}, \mathbf{b}) := \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{d} b_j e^{-a_j x_i} \right)^2,$$

over two parameters $\mathbf{a}$ and $\mathbf{b}$. Given parameters $\mathbf{a}$, describe how we can minimize over $\mathbf{b}$. That is, how can we compute,

$$\hat{\mathbf{b}} := \arg\min_{\mathbf{b}} J(\mathbf{a}, \mathbf{b}).$$

(c) In the above example, how would we compute the gradients,

$$\nabla_{\mathbf{a}} J(\mathbf{a}, \mathbf{b}).$$

Solution:

(a) By definition, $J_1(\mathbf{w}_1) = J(\mathbf{w}_1, \widehat{\mathbf{w}}_2(\mathbf{w}_1))$ where $\widehat{\mathbf{w}}_2(\mathbf{w}_1) = \arg\min_{\mathbf{w}_2} J(\mathbf{w}_1, \mathbf{w}_2)$.
Using the chain rule:

$$\nabla_{\mathbf{w}_1} J_1(\mathbf{w}_1) = \nabla_{\mathbf{w}_1} J(\mathbf{w}_1, \widehat{\mathbf{w}}_2(\mathbf{w}_1)) + \nabla_{\mathbf{w}_2} J(\mathbf{w}_1, \widehat{\mathbf{w}}_2(\mathbf{w}_1)) \frac{\partial \widehat{\mathbf{w}}_2}{\partial \mathbf{w}_1}$$

However, since $\widehat{\mathbf{w}}_2(\mathbf{w}_1)$ minimizes $J(\mathbf{w}_1, \mathbf{w}_2)$ over $\mathbf{w}_2$, we have:

$$\nabla_{\mathbf{w}_2} J(\mathbf{w}_1, \widehat{\mathbf{w}}_2(\mathbf{w}_1)) = 0$$

Therefore:

$$\nabla_{\mathbf{w}_1} J_1(\mathbf{w}_1) = \nabla_{\mathbf{w}_1} J(\mathbf{w}_1, \widehat{\mathbf{w}}_2(\mathbf{w}_1))$$

This is the *envelope theorem* - the gradient of the optimal value function equals the gradient of the original function with respect to the remaining variables, evaluated at the optimal solution.

(b) Given $\mathbf{a}$, the problem becomes:

$$\hat{\mathbf{b}} = \arg\min_{\mathbf{b}} \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{d} b_j e^{-a_j x_i} \right)^2$$

This is a linear least squares problem in $\mathbf{b}$. Let:

$$A_{ij} = e^{-a_j x_i}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_d \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Then the problem becomes:

$$\hat{\mathbf{b}} = \arg\min_{\mathbf{b}} \|\mathbf{y} - A\mathbf{b}\|^2$$

The solution is:

$$\hat{\mathbf{b}} = (A^T A)^{-1} A^T \mathbf{y}$$

(c) To compute $\nabla_{\mathbf{a}} J(\mathbf{a}, \mathbf{b})$, we use the envelope theorem from part (a):

$$\nabla_{\mathbf{a}} J(\mathbf{a}, \mathbf{b}) = \nabla_{\mathbf{a}} J(\mathbf{a}, \mathbf{b})|_{\mathbf{b}=\hat{\mathbf{b}}(\mathbf{a})}$$

where $\hat{\mathbf{b}}(\mathbf{a})$ is the optimal $\mathbf{b}$ for given $\mathbf{a}$.
The gradient with respect to $\mathbf{a}$ is:

$$\frac{\partial J}{\partial a_j} = \sum_{i=1}^{n} 2 \left(y_i - \sum_{k=1}^{d} b_k e^{-a_k x_i} \right) \cdot b_j x_i e^{-a_j x_i}$$

Evaluated at $\mathbf{b} = \hat{\mathbf{b}}(\mathbf{a})$:

$$\nabla_{\mathbf{a}} J(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^{n} 2 \left(y_i - \sum_{k=1}^{d} \hat{b}_k e^{-a_k x_i} \right) \cdot \hat{b}_j x_i e^{-a_j x_i}$$

This gives us the gradient for optimizing $\mathbf{a}$ using nested optimization.